

AI Multi-Asset Trading System

Project Workflow & Architecture Document

Assets: Bitcoin (BTC) | Gold (XAU) | Silver (XAG)

Exchange: Bybit V5 API (Testnet Execution / Mainnet Data)

GitHub: github.com/ayush39681016/ai-dtrading

Dashboard: Streamlit (Local + Cloud Deployed)

Date: April 2026

1. System Architecture Overview

The system is a self-improving, multi-asset algorithmic trading platform. It combines real-time market data, technical analysis, automated order execution, and a visual dashboard into an integrated pipeline.

Core Pipeline

- DATA FEED: Bybit V5 Mainnet API fetches 5-min OHLCV candles (500 per asset)
- INDICATORS: EMA(fast/slow), RSI(14), ATR(14), Volume MA(20), EMA(200)
- SIGNAL ENGINE: Confluence of EMA crossover + RSI filter + Volume confirmation + Trend filter
- EXECUTION: Bybit Testnet API places market orders with ATR-based SL/TP
- LOGGING: Every trade is recorded in trades.csv with timestamp, symbol, and strategy version
- DASHBOARD: Streamlit app displays live prices, equity curve, backtest panels, and trade log
- AUTO-OPTIMIZER: Every 20 trades, checks performance and triggers grid-search re-optimization

Project Files (17 files on GitHub)

File	Purpose
strategy_engine.py	Main loop - scans 3 assets every 60s, generates signals, executes trades
app.py	Streamlit dashboard - live prices, backtest panels, trade log, equity curve
optimizer.py	Grid-search across 945 param combos with 70/30 train/test split
backtester.py	Historical backtesting engine with PnL calculation
executor.py	Bybit V5 API wrapper for market orders, balance queries
data_cache.py	Downloads & caches 1000 candles from Mainnet for offline optimization
performance.py	Per-symbol trade pairing, win rate, profit factor, drawdown analytics
force_buy.py	Manual test trade trigger for validation
trade_logger.py	Trade logging utility
webhook_server.py	Webhook endpoint for external alerts
test_bybit.py	API connection tester
install.py	Dependency installer
config.json	Optimized strategy parameters (auto-generated)
requirements.txt	Python package dependencies
Procfile	Cloud deployment process definitions
railway.toml	Railway.app deployment configuration
start_system.bat	Windows one-click launcher

2. Trading Strategy

Entry Rules (BUY)

- EMA(fast) crosses ABOVE EMA(slow) [Trend confirmation]
- RSI(14) < 65 [Not overbought]
- Current Volume > 20-period Volume MA [Volume confirmation]
- Price > EMA(200) [Long-term uptrend filter]

Exit Rules (SELL)

- EMA(fast) crosses BELOW EMA(slow) [Trend reversal signal]
- Price hits Stop Loss = Entry - ATR x 1.5 [Risk management]
- Price hits Take Profit = Entry + ATR x 3.0 [Reward target]

Risk Management

- ATR-based dynamic stop loss and take profit
- Independent position tracking per asset (BTC, Gold, Silver)
- Auto re-optimization triggered every 20 trades if performance degrades
- Train/test split (70/30) prevents overfitting in optimizer

Optimization Engine

The optimizer performs a grid search across 945 parameter combinations. Parameters tested: EMA fast (5-20), EMA slow (15-50), RSI threshold (55-75), ATR SL multiplier (1.0-2.0), ATR TP multiplier (2.0-4.0).

Filters: Min Profit Factor > 1.5, Min Win Rate > 50%, Max Drawdown < 25%, Min 3 Trades. Top combos are validated on out-of-sample test data before the best is saved to config.json.

3. Deployment & Infrastructure

Platform	URL	Purpose
GitHub	github.com/ayush39681016/ai-dtrading	Code repository & collaboration
Localhost	http://localhost:8520	Local development dashboard
Streamlit Cloud	ai-dtrading-...streamlit.app	Cloud dashboard (free, always-on)
Railway.app	railway.com project	Cloud bot + dashboard (30-day trial)

Technology Stack

- Language: Python 3.13
- Exchange API: Bybit V5 (REST)
- Dashboard: Streamlit
- Data: Pandas, NumPy
- Visualization: Matplotlib
- Deployment: Railway.app, Streamlit Cloud, GitHub

4. Development Timeline

Phase	Name	What Was Done
Phase 1	Single-Asset BTC Bot	Built EMA+RSI strategy for BTCUSDT on Bybit Testnet with basic execution
Phase 2	Bybit API Integration	Connected to Bybit V5 API, resolved WAF/IP blocks with mock fallback
Phase 3	Auto-Optimization	Grid-search optimizer with 945 param combos + train/test validation
Phase 4	Dashboard v1	Streamlit dashboard with equity curve, trade log, performance metrics
Phase 5	Multi-Asset Expansion	Added Gold (XAUUSDT) and Silver (XAGUSDT) with per-symbol tracking
Phase 6	Offline Data Cache	Implemented data_cache.py for instant backtesting without API calls
Phase 7	Dashboard v2	Asset selector, instant backtest panels, live Mainnet price tickers
Phase 8	System Audit & Fixes	Fixed 7 critical bugs: dead imports, indentation, interval mismatches
Phase 9	Data Accuracy	Switched data sources from Testnet to Mainnet for real-world prices
Phase 10	GitHub & Cloud	Pushed to GitHub, deployed on Streamlit Cloud + Railway.app

Bugs Fixed in System Audit

- Removed dead 'binance.client' import (system uses Bybit)
- Fixed signal logic running outside the per-symbol for-loop
- Updated CSV column format for multi-asset trade logging
- Added empty-DataFrame guards to prevent IndexError crashes
- Fixed optimizer interval mismatch ('1h' -> '60')
- Updated force_buy.py to new 4-argument log_trade signature
- Switched backtester + data_cache from Testnet to Mainnet

5. Strategic Roadmap (Next Steps)

Phase 1: Improve Strategy Edge (Priority: NOW)

- Add Market Regime Detection (ADX-based: trending vs choppy)
- Add Mean Reversion strategy for ranging markets
- Implement risk-based position sizing (0.5-1% of account per trade)

Phase 2: Institutional-Grade Signals

- Funding Rate filter (fade overcrowded trades)
- Open Interest spikes (continuation confirmation)
- Cross-exchange spread monitoring

Phase 3: Portfolio Risk Management

- Daily loss limit (-2% circuit breaker)
- Correlation monitor across assets
- Trailing stop loss (move SL to breakeven after 1x ATR move)
- Partial profit taking (50% at 1.5x ATR, 50% at 3x ATR)

Phase 4: Proper Backtesting

- Include transaction costs (0.06% maker + 0.1% taker)
- Model slippage (0.02-0.05% per trade)
- Walk-forward testing instead of single train/test split
- Monte Carlo simulation for robustness validation

Phase 5: Production Deployment

- Deploy on a cloud VPS for 24/7 autonomous operation
- Paper trade for 2-4 weeks before going live
- Start with minimum position sizes (\$10-50 per trade)
- Add Telegram alerts for real-time trade notifications
- Scale up only after 100+ live trades with positive expectancy